

Women360 — Review 2 Report

BCSE408L — Cloud Computing Project: Women360 — a cloud-native women's health & wellness SaaS platform

Team member 1	Pavithra Uthrah R K - 23BCE1818
Team member 2	Asmi Singh
GitHub repository	https://github.com/uthrah/Women360
Live frontend	https://women-360.vercel.app
Live API	https://women360-api.onrender.com

This report documents Women360's system architecture, cloud architecture, database design, module design, API design, authentication model, and the initial cloud deployment, per the Review 2 deliverable checklist. Supporting diagrams referenced throughout are in this same [Documentation/](#) folder.

1. System Architecture

Women360 is built as a **monorepo with two independently deployable applications**, connected only over HTTP:

```
Women360/
├── src/
│   ├── components/ui/      Frontend – React 18 + TypeScript + Vite SPA
│   │                       Design-system primitives (Button, Card, Modal, Toast, states...)
│   ├── components/layout/  App shell: sidebar, bottom nav, Senior Mode navigation
│   ├── features/           One folder per product area (dashboard, cycle, nutrition,
│   │                       activity, sleep, wellbeing, goals, health, insights,
│   │                       reports, learn, messages, settings, onboarding, auth)
│   ├── services/           One file per domain; the ONLY layer that talks to the API
│   ├── context/, hooks/    App-wide auth/theme/Senior-Mode state
│   └── types/              Shared frontend domain types
├── server/
│   └── src/
│       ├── modules/        Backend – Node.js + TypeScript + Express + Prisma API
│       │                       One folder per domain (see $5 Module Design)
│       ├── middleware/     requireAuth, requireRole (RBAC), validate (zod), rate limiting
│       ├── lib/            Prisma client, JWT signing, password hashing, response envelope
│       └── config/         Validated environment configuration
└── prisma/schema.prisma    Single source of truth for the relational data model
```

Request flow (a typical write, e.g. logging a meal):

```
Browser (React component)
├── src/services/nutritionService.ts      (domain method call)
├── src/services/apiClient.ts request()   (adds Bearer token, JSON envelope)
├── HTTPS
├── Express route (server/src/modules/nutrition/nutrition.routes.ts)
├── requireAuth middleware                 (verifies JWT, attaches req.user)
├── validate middleware                    (zod schema on the request body)
├── nutrition.service.ts                  (business logic, ownership checks)
├── Prisma Client → PostgreSQL
└── JSON envelope { ok, data | error } back through the same chain
```

Every layer has a single, narrow responsibility: **routes** wire HTTP to handlers and declare validation; **validation** (zod) rejects malformed input before it reaches business logic; **services** hold the actual logic and are the only layer that talks to Prisma; **Prisma** is the only layer that talks to SQL. The frontend mirrors this discipline — no component calls `fetch` directly; everything goes through a `services/*.ts` file, which is what allowed the entire backend to be built and swapped in later without changing a single page.

A shared `mappers.ts` module on the frontend is the one place that translates between the backend's `UPPER_SNAKE_CASE` enums and the frontend's lowercase domain vocabulary, so the translation logic exists exactly once.

(See [System Architecture Diagram.png](#) in this folder for the visual component diagram.)

2. Cloud Architecture

Women360 is deployed across three managed cloud services, chosen deliberately within the SRS's approved list (\$4: Postgres/Mongo Atlas/Firebase/Supabase for the database; Render/Railway/AWS/Azure/GCP/Firebase for hosting):

Layer	Provider	Service type	Why
Frontend	Vercel	Static SPA hosting + global CDN	Free tier, git-integrated auto-deploy on every push, purpose-built for Vite/React
API	Render	Node.js web service	Free tier, git-integrated auto-deploy, runs a persistent long-lived Node process (not serverless functions)
Database	Supabase	Managed PostgreSQL	Free tier has no hard expiry (Render's own free Postgres expires after 30 days, which would kill the database mid-semester); an SRS-approved option in its own right



Deployment pipeline: both Vercel and Render are connected directly to the `main` branch of the GitHub repository. Every push triggers an independent build and deploy on each platform — there is no manual upload step. The API's build step (`server/render.yaml`, a Render Blueprint) runs `npm ci --include=dev && npx prisma generate && npm run build && npx prisma migrate deploy`, meaning **every schema migration is applied automatically to production on deploy**, not as a separate manual step.

Configuration is environment-driven, never hardcoded: the API reads `DATABASE_URL`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, and `CORS_ORIGINS` from Render's environment variables (validated at boot with zod — the process refuses to start on invalid config); the frontend reads `VITE_API_BASE_URL` from Vercel's environment variables, inlined into the static bundle at build time.

(See [Cloud Architecture & Deployment Diagram.png](#) for the visual deployment diagram.)

3. Database Design

PostgreSQL via Prisma ORM, **25 models** across eight domains, all scoped to a `User` by a `userId` foreign key with `onDelete: Cascade`:

Domain	Models
Identity & access	<code>User</code> , <code>RefreshToken</code> , <code>PasswordResetToken</code>
Profile & preferences	<code>HealthProfile</code> , <code>EmergencyContact</code> , <code>SeniorEssentialPreference</code>
Menstrual & reproductive health	<code>CycleProfile</code> , <code>CycleEntry</code>
Nutrition	<code>NutritionGoal</code> , <code>MealEntry</code> , <code>HydrationLog</code> , <code>FruitVegLog</code>
Activity & sleep	<code>ActivityGoal</code> , <code>ActivityEntry</code> , <code>StepsLog</code> , <code>SleepEntry</code>
Wellbeing & goals	<code>WellbeingEntry</code> , <code>Goal</code>
Preventive care	<code>Appointment</code> , <code>Medication</code> , <code>VitalMeasurement</code>
Reports, messaging, learning	<code>Report</code> , <code>Notification</code> , <code>Message</code> , <code>LearnArticle</code>
Coach & administration	<code>CoachAssignment</code> , <code>CoachNote</code> , <code>AuditLog</code>

Design decisions worth noting:

- Role-based identity, not separate tables per role.** `User.role` (`WOMAN` / `COACH` / `ADMIN`) is a single enum column, enforced server-side by a `requireRole` middleware on every protected route — never trusted from the client.
- Goal progress is never stored as a percentage.** `Goal.currentValue` and `Goal.targetValue` are real numbers with a `unit` string (e.g. "8000 steps"); the percentage shown anywhere in the app is always computed as `currentValue / targetValue`, so it can never drift out of sync with the underlying numbers.
- Sleep quality is a 1–5 rating** (`SleepEntry.quality`, 1 Very poor .. 5 Excellent), not a 0–100 percentage — a deliberate scale choice to keep the metric intuitive rather than falsely precise.
- Nutrition totals are never separately cached.** `MealEntry.calories/proteing/carbsG/fatG/fibreG` are all `Float` (supporting one decimal place); a day's totals are always summed live from the actual meal rows at read time, so they can never disagree with what was logged.
- Coach access is opt-in and audited.** A `CoachAssignment` only exists once a woman explicitly grants it; every privileged/administrative action is written to `AuditLog` (structural metadata only — action, actor, target — never the health record contents themselves).

(See [ER- Diagram.png](#) for the full entity-relationship diagram.)

4. Module Design

The backend has **16 domain modules** under `server/src/modules/`, each following an identical three-file shape:

```

<domain>/
├── <domain>.routes.ts    Express router: wires HTTP verbs + paths to handlers,
│                         declares which zod schema validates the request
├── <domain>.service.ts   Business logic: the only layer that calls Prisma;
│                         owns ownership checks (a user can only touch their own rows)
└── <domain>.validation.ts zod schemas for request bodies/params/queries
  
```

Modules: `auth`, `users`, `cycle`, `nutrition`, `activity`, `sleep`, `wellbeing`, `goals`, `health`, `insights`, `reports`, `learn`, `messages`, `notifications`, `coach`, `admin`.

This uniform shape means adding a new domain is mechanical (a Prisma model + three files + one line in `router.ts`), and it's what let four modules (Nutrition, Sleep, Cycle, Goals) be taken from read-only mockups to full CRUD without touching any other module's code.

(See [Backend Module Design & Component Diagram.png](#) for the component diagram.)

5. API Design

All endpoints are namespaced under `/api/v1`, return a consistent JSON envelope (`{ "ok": true, "data": ... }` or `{ "ok": false, "error": { "code", "message" } }`), and — except `/auth/register`, `/auth/login`, `/auth/refresh`, and the password-reset endpoints — require a valid `Authorization: Bearer <token>` header.

Module	Method & path	Purpose
auth	POST <code>/auth/register</code>	Create account, return session
	POST <code>/auth/login</code>	Authenticate, return session
	POST <code>/auth/refresh</code>	Rotate access/refresh token pair
	POST <code>/auth/logout</code>	Revoke a refresh token server-side
	POST <code>/auth/password-reset/request</code>	Request a reset token (enumeration-safe)
	POST <code>/auth/password-reset/confirm</code>	Consume a reset token
	POST <code>/auth/onboarding/complete</code>	Mark onboarding done
	GET <code>/auth/me</code>	Current authenticated user
users	PATCH <code>/users/me</code>	Update profile
	GET/PUT <code>/users/me/emergency-contact</code>	Emergency contact
	GET/PATCH <code>/users/me/senior-essentials</code>	Senior Mode tile selection
	GET/PUT <code>/users/me/health-profile</code>	Allergies/conditions
cycle	GET <code>/cycle/summary</code>	Current day/phase/prediction
	GET <code>/cycle/entries</code>	History
	PUT/DELETE <code>/cycle/entries/:date</code>	Log/edit/remove a day
	GET/PUT <code>/cycle/profile</code>	Average cycle/period length
nutrition	GET <code>/nutrition/today</code>	Daily summary (live-aggregated)
	POST/PATCH/DELETE <code>/nutrition/meals[:id]</code>	Meal CRUD
	POST <code>/nutrition/hydration</code>	Log water
	POST <code>/nutrition/fruit-veg</code>	Log a serving
	GET/PUT <code>/nutrition/goal</code>	Nutrition goals
activity	GET <code>/activity/summary</code>	Weekly summary
	POST/DELETE <code>/activity/entries[:id]</code>	Activity CRUD
	POST <code>/activity/steps</code> , PUT <code>/activity/goal</code>	Steps, goals
sleep	GET <code>/sleep/summary</code>	Weekly summary + history
	PUT/DELETE <code>/sleep/entries/:date</code>	Log/edit/remove a night
wellbeing	GET <code>/wellbeing/entries</code> , PUT <code>/wellbeing/entries/:date</code>	Mood/stress/energy check-ins
goals	GET/POST <code>/goals</code> , PATCH/DELETE <code>/goals/:id</code>	Goal CRUD
health	GET/POST/PATCH/DELETE <code>/health/appointments[:id]</code>	Appointments
	GET/POST/PATCH/DELETE <code>/health/medications[:id]</code>	Medications
	GET/POST <code>/health/vitals</code>	Vitals
insights	GET <code>/insights</code>	Derived, non-diagnostic trends
reports	GET <code>/reports</code> , GET <code>/reports/:id</code> , POST <code>/reports</code>	Health report generation
learn	GET/POST/PATCH/DELETE <code>/learn[:id]</code>	Educational content
messages	GET <code>/messages</code> , PATCH <code>/messages/:id/read</code>	Messaging
notifications	GET <code>/notifications</code> , PATCH <code>/notifications/:id/read</code>	Notifications
coach	POST/DELETE/GET <code>/coach/sharing</code> , GET <code>/coach/assigned-women</code> , GET/POST <code>/coach/women/:womanId/*</code>	Coach ↔ woman data sharing
admin	GET <code>/admin/users</code> , PATCH <code>/admin/users/:userId/role</code> , DELETE <code>/admin/users/:userId</code> , GET <code>/admin/audit-log</code>	Administration (role-gated)

6. Authentication

- **Password storage:** Argon2id (OWASP's current recommendation) — never plaintext, never a reversible cipher.
- **Session model:** short-lived **JWT access tokens** (15 minutes, stateless, carry `{sub, role}`) plus long-lived, opaque, random **refresh tokens** (30 days). Only the SHA-256 hash of a refresh token is ever stored in the database — a stolen database dump alone cannot be replayed as a live session.

- **Rotation:** every use of a refresh token immediately revokes it and issues a new pair, so a stolen-but-already-used refresh token is a dead end for an attacker.
- **Identity is never trusted from the client.** Every protected route derives the acting user from the verified JWT (`requireAuth` middleware sets `req.user`), never from a request body or query parameter — confirmed by a full-codebase audit that found zero routes trusting a client-supplied user ID.
- **Authorization is enforced twice:** `requireRole` gates entire route groups (e.g. `/admin/*`), and every service method that reads/writes a specific record (a goal, a meal, a report) independently checks that record's `userId` matches the authenticated user before allowing access — verified by automated tests that register two users and confirm one cannot read, edit, or delete the other's data (`403 Forbidden`).
- **No account enumeration:** login and password-reset failures return one identical, generic message regardless of whether the email exists or the password was wrong.
- **Logout is real, not cosmetic:** it revokes the refresh token's database row, so the session cannot be resumed even if the token value leaked.

(See `Authentication & RBAC Sequence Diagram.png` for the sequence diagram.)

7. Core Modules (functional depth)

Four modules were taken to full, tested CRUD depth as the functional centerpiece of this phase:

- **Nutrition** — meal logging with calories/protein/carbs/fat/fibre (one decimal place), live daily totals aggregated directly from that day's actual records (not a separately-cached counter), hydration and fruit-&-veg quick-logging, full edit/delete.
- **Sleep** — bedtime/wake-time logging with automatic overnight-crossing duration calculation, a 1–5 quality rating (never a percentage), weekly trend chart, full edit/delete.
- **Cycle** — calendar-based period/flow/symptom logging, a derived (never fabricated) current-phase and next-period estimate based on the user's own logged history, explicitly labeled as an estimate rather than a diagnosis.
- **Goals** — real current/target values with a unit, live-computed progress, active/completed views, manual progress updates and completion toggling.

Every one of these has automated integration tests that exercise it through the real HTTP API against a real database (registering a user, performing the CRUD sequence, and asserting on the actual stored/returned values) — not just unit tests against mocked data.

8. GitHub Repository

<https://github.com/uthrahh/Women360>

- Single monorepo, `main` branch, regular incremental commits with descriptive messages (not one bulk upload).
- Both frontend (`root`) and backend (`server/`) have their own linting, TypeScript strict-mode typechecking, and automated test suites (44 backend integration/unit tests, 12 frontend unit tests, all currently passing).
- `render.yaml` and `vercel.json` are committed at the repo root, so the entire cloud deployment configuration is version-controlled alongside the code, not configured only by hand in each provider's dashboard.

9. Initial Cloud Deployment

Frontend (live)	https://women-360.vercel.app
API (live)	https://women360-api.onrender.com
API health check	https://women360-api.onrender.com/health
Database	Supabase-managed PostgreSQL (private connection string, not public)

Verified working end-to-end (not just "the build succeeded"): a real account can be registered through the live frontend, receives a real JWT session from the live API, and that session is persisted in a real Supabase-hosted Postgres database — confirmed by directly testing signup, login, and session persistence against the production URLs above, not just localhost.

A note on the free tier: Render's free web service spins down after 15 minutes of inactivity and takes roughly 30–50 seconds to wake up on the next request — expected behavior for a free-tier demo, not a defect. This is disclosed here rather than left as a surprise during a live demo.

Appendix: Diagram index

File	Covers
<code>System Architecture Diagram.png</code>	\$1 System Architecture
<code>Cloud Architecture & Deployment Diagram.png</code>	\$2 Cloud Architecture
<code>ER- Diagram.png</code>	\$3 Database Design
<code>Backend Module Design & Component Diagram.png</code>	\$4 Module Design
<code>Authentication & RBAC Sequence Diagram.png</code>	\$6 Authentication
<code>Use Case Diagram.png</code>	Carried over from Review 1 — functional scope